1. S: Single Responsibility Principle (SRP)

"A class should have one, and only one, reason to change."

- **The Problem:** Imagine a class Report. It calculates data, formats it into a PDF, and then emails it. If the email server changes, you have to modify the Report class. If the PDF layout changes, you modify the same class. This is "God Object" behavior.
- **The Example:**
    - ReportDataRetriever: Only fetches data.
    - ReportFormatter: Only converts data into a specific format (PDF/CSV).
    - EmailService: Only handles sending messages.
- **Benefit:** If you want to switch from Email to Slack notifications, you only touch the EmailService (or create a SlackService), leaving your data logic untouched.

---

2. O: Open/Closed Principle (OCP)

"Software entities should be open for extension, but closed for modification."

- **The Problem:** You have a DiscountCalculator with an if/else block: if (customer == "Premium") { 20% } else if (customer == "VIP") { 30% }. Every time a new customer tier is added, you must **modify** this existing, tested class, risking new bugs.
- **The Example:**
    - Create an interface DiscountStrategy with a method apply().
    - Create classes PremiumDiscount and VIPDiscount that implement that interface.
    - The DiscountCalculator simply calls strategy.apply().
- **Benefit:** To add a "Holiday Discount," you just create a new class. You never touch the original DiscountCalculator.

---

3. L: Liskov Substitution Principle (LSP)

"Subtypes must be substitutable for their base types without altering the correctness of the program."

- **The Problem:** The classic "Square-Rectangle" problem. If a Square inherits from Rectangle, but setting the width also changes the height, it breaks the expectation of what a Rectangle does.
- **The Example:**
    - Suppose you have a base class Bird with a fly() method.
    - If you create a class Ostrich that inherits from Bird, but ostriches can't fly, you might throw an UnsupportedOperationException. This violates LSP because a program expecting a Bird will crash when it gets an Ostrich.
- **The Fix:** Don't put fly() in the base class. Instead, have a Bird base class and a separate interface called Flyable.

---

4. I: Interface Segregation Principle (ISP)

"A client should never be forced to implement an interface that it doesn't use."

- **The Problem:** You have a "fat" interface called SmartDevice with methods: print(), fax(), and scan().
- **The Issue:** If you have a simple InkjetPrinter, it is forced to implement fax() and scan() even if it doesn't have those features, usually leaving them empty or throwing errors.
- **The Example:**
    - Break the fat interface into Printer, FaxMachine, and Scanner.
    - An AllInOneOfficePro implements all three.
    - A BasicPrinter only implements Printer.
- **Benefit:** Clients only depend on the methods they actually need.

---

5. D: Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

- **The Problem:** A Notification class that directly creates a GmailService inside its constructor. The high-level class (Notification) is "stuck" with the low-level class (Gmail).
- **The Example:**
    - Define an interface MessageService.
    - Notification class takes MessageService as a constructor argument (Dependency Injection).
    - At runtime, you can "inject" GmailService, SmsService, or MockEmailService for testing.
- **Benefit:** You can swap out your database or your messaging provider without changing a single line of your business logic.

Putting it into the "Batching" Context

If you were building your **Log Batcher** using SOLID:

1. **SRP:** One class collects logs, another class sends the batch.
2. **OCP:** You can add new "Batch Destinations" (Database, Cloud, File) without changing the Batcher.
3.